МИРЭА- Российский технологический университет

Институт перспективных технологий и индустриального программирования

Дисциплина

«Алгоритмы машинного обучения для решения прикладных задач»

Семестр 1

Практическое занятие №7-8

Практическая работа стохастический градиентный спуск. Практическая работа алгоритм оптимизации Adam и Adamgrad.

Доцент кафедры индустриального программирования
к. ф.-м. н.
Осипов Алексей Викторович

Москва, 2026 г.

Практическая работа: "Бинарная логистическая регрессия"

Задача бинарной классификации

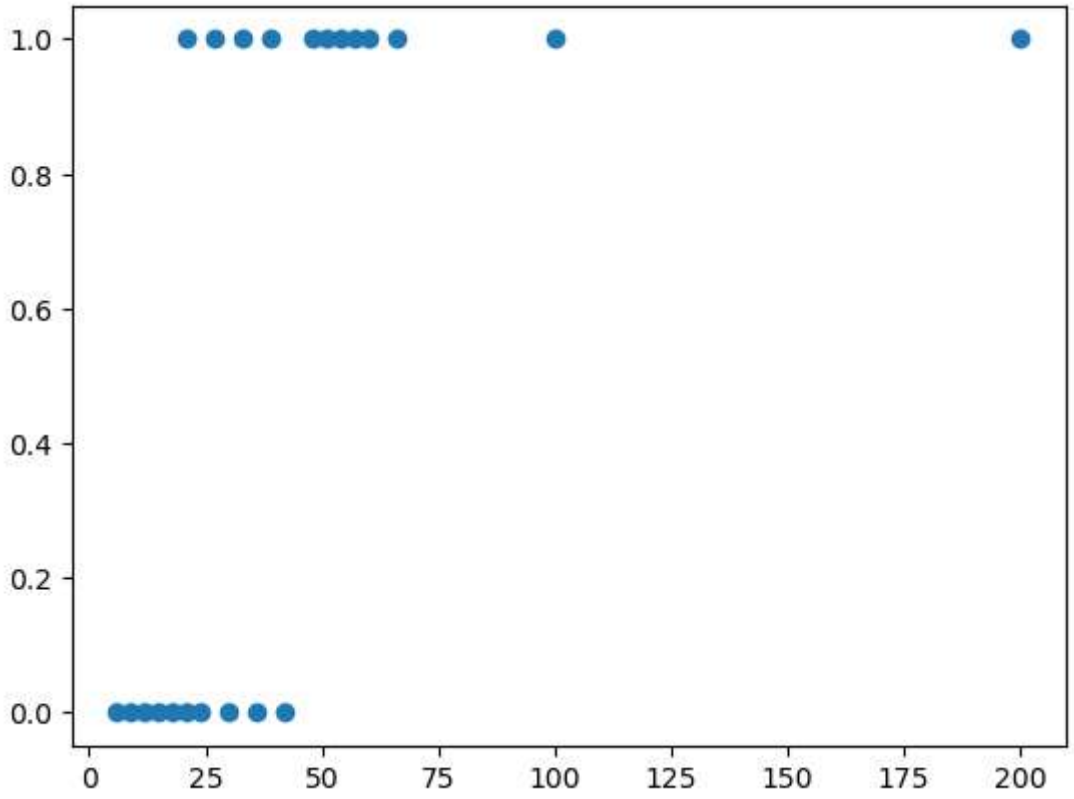
Некая фирма провела анализ зависимости расхода денежных средств на рекламу и эффекта от нее. Если эффект присутствовал, то в датасете ставилась 1, в противном случае - 0. Результаты были представоены в виде двух DataFrame x и y.
Произыести бинарную классификацию с использованием логистической регрессии.

In [1]:
```
#Загрузим необходимые библиотеки
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

In [3]:
```
#Исходные данные: x - расходы на рекламу, Y - объемы продаж
x = pd.DataFrame(np.array([39, 36, 60, 21, 27, 12, 51, 57, 54, 42, 18, 9, 48, 66, 6, 30, 24, 33, 15, 21, 100,
Y = pd.DataFrame(np.array([1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1]))
#display(x, Y)
```

In [4]:
```
plt.scatter(x,Y)
```

Out[4]:
```
<matplotlib.collections.PathCollection at 0x29e6dd0dbe0>
```



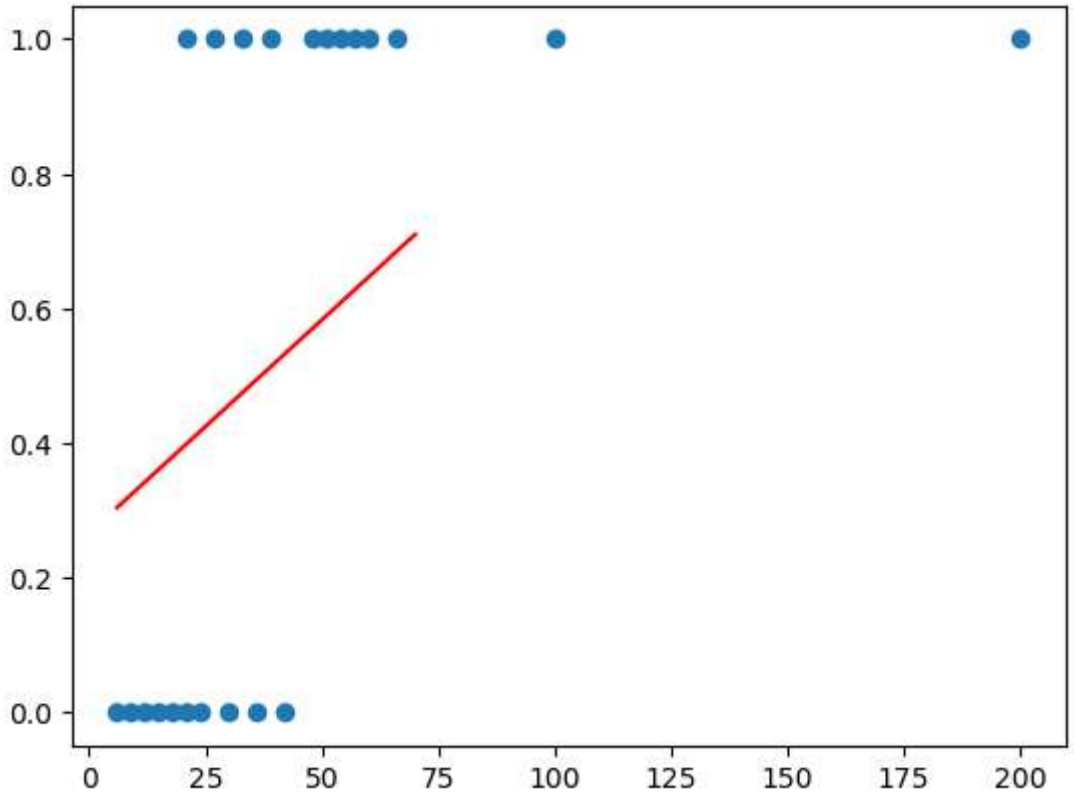
Попробуем разделить эти точки прямой линией, отделяющей один класс от другого. Модель начинает плохо работать в случае добавления новых точек на больших значениях x:

In [5]:
```
a1 = ((x-x.mean())*(Y-Y.mean())).mean()/((x-x.mean())**2).mean()
a0 = Y.mean() - a1*x.mean()
print("Y = ", a0, " + ", a1, " * x")
x_space = np.linspace(6,70,20)
Y_pred = a0.item() + a1.item()*x_space
```

```
Y =  0    0.266493
dtype: float64  +  0    0.006333
dtype: float64  * x
```

In [6]:
```
plt.plot(x_space, Y_pred, 'r')
plt.scatter(x, Y)
```

Out[6]: <matplotlib.collections.PathCollection at 0x29e6e42ddf0>

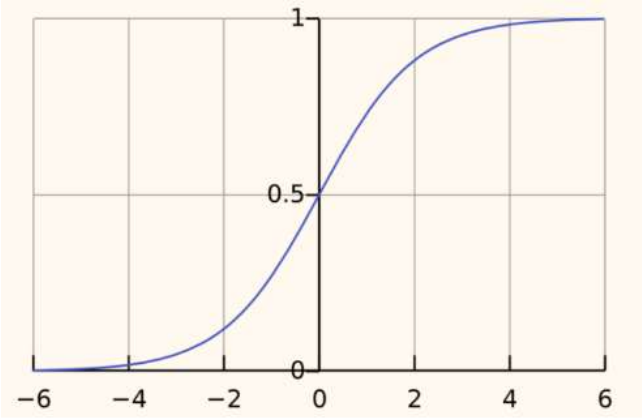


Функция логистической регрессии

Сигмоида

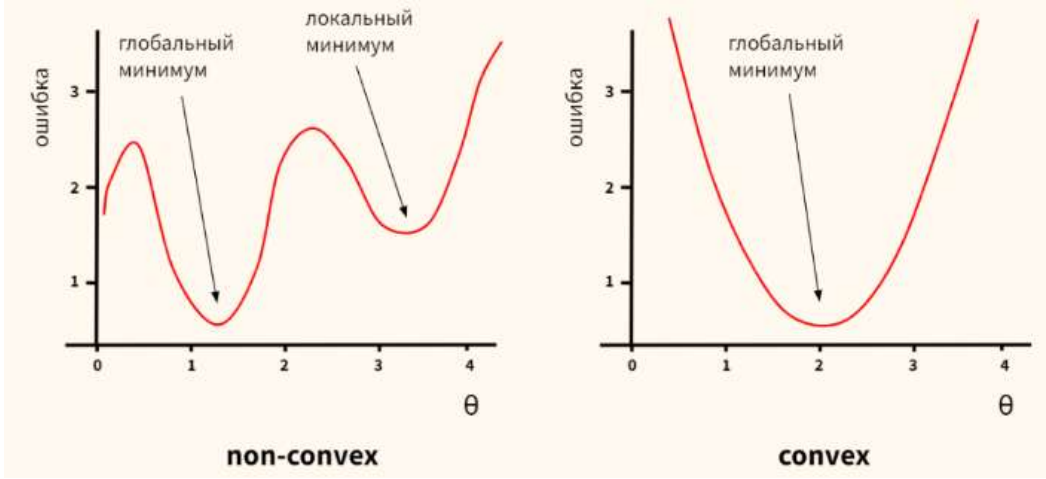
Возможное решение упомянутых выше сложностей — пропустить значение линейной регрессии через сигмоиду (sigmoid function), которая при любом значении x не выйдет из необходимого нам диапазона от 0 до 1.

$$g(x) = \frac{1}{1+e^{-x}}$$



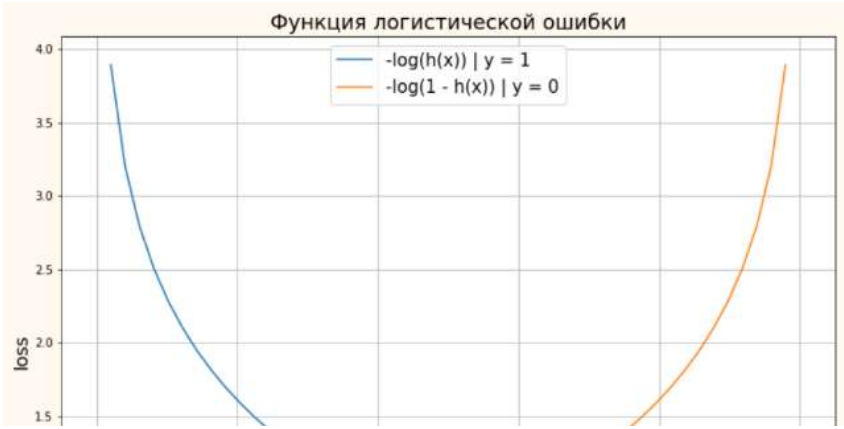
Logistic loss или функция кросс-энтропии

В модели логистической регрессии мы не можем использовать MSE. Дело в том, что если мы поместим результат сигмоиды (представляющей собою нелинейную функцию) в MSE, то на выходе получим невыпуклую функцию (non-convex), глобальный минимум которой довольно сложно найти.



Вместо MSE мы будем использовать функцию логистической ошибки, которую еще называют функцией бинарной кросс-энтропии (log loss, binary cross-entropy loss).

График и формула функции логистической ошибки



$$J(\theta) = -\frac{1}{n} \sum y * log(h_\theta(x)) + (1 - y) * log(h_\theta(x))$$

Оценка качества модели

Матрица ошибок (confusion matrix)

	Прогноз: отрицательный (predicted negative)	**Прогноз: положительный** (predicted positive)
Факт: отрицательный (actual negative)	Истинно отрицательный (true negative, TN)	Ложноположительный (false positive, FP)
Факт: положительный (actual positive)	Ложноотрицательный (false negative, FN)	Истинно положительный (true positive, TP)

Доля правильно предсказанных значений называется accuracy. Чтобы ее посчитать, мы берем те значения, которые предсказаны верно (TP + TN) и делим на общее количество прогнозов.

$$TP + TN$$

Конкретизация задачи:

Реализовать класс модели логистической регрессии с использованием:

а) максимизации логарифмического правдоподобия методом градиентного подъема

б) минимизации функции потерь (кросс-энтропии) методом градиентного спуска

Сравнить результаты (время исполнения и достигнутые значения accuracy и f1) в случаях а и б при одном и том же числе шагов и скорости обучения.

In [7]:
```
#Реализуем цикл градиентного спуска
class LogisticRegressionGD(object):
    def __init__(self):
        self.a = np.zeros(2).reshape(1, 2)

    def sigmoid(self, x):
        return 1/(1 + np.exp(-x @ self.a))

    def predict(self, x):
        return self.sigmoid(x)

    def coefs(self):
        return self.a

    def LogLikelihood(self, x, Y):
        return (Y * np.log(self.predict(x)) + (1 - Y) * np.log(1 - self.predict(x))).sum()

    def CrossEntropy(self, x, Y):
        return (-Y * np.log(self.predict(x)) - (1 - Y) * np.log(1 - self.predict(x))).sum()

    def accuracy(self, x, Y):
        return ((self.predict(x) > 0.5)==Y).mean()

    def fit(self, x, Y, alpha = 0.01, epsylon = 0.01, max_steps = 10000, Rtype = "LL"):
        self.a = np.zeros (x.shape[1]).reshape(x.shape[1],1)
        steps, errors = [], []
        step = 0
        for _ in range(max_steps):
            if Rtype == "LL":
                new_error = self.LogLikelihood(x, Y)
                dT_a = x.T @(Y - self.predict(x)) / x.shape[0]
                self.a += alpha*dT_a
            elif Rtype == "CE":
                new_error = self.CrossEntropy(x, Y)
                #display(new_error)
                dT_a = -x.T @(Y - self.predict(x)) / x.shape[0]
                self.a -= alpha*dT_a
            step += 1
            steps.append(step)
            errors.append(new_error)
            #if new_error < epsylon:
            #    break
```

```
        return steps, errors
```

In [8]:
```
x_ = x.copy()

import time

intercept = np.ones((x.shape[0], 1))
x_ = pd.DataFrame(np.concatenate((intercept, x), axis = 1))
```

In [117]:
```
start_time_CE = time.time()
regr_CE = LogisticRegressionGD()
steps_CE, errors_CE =  regr_CE.fit(x_, Y, alpha = 0.01, epsylon = 0.01, max_steps = 10000, Rtype = "CE")

display(f"Время обучения: {time.time() - start_time_CE} сек.")

start_time_LL = time.time()
regr_LL = LogisticRegressionGD()
steps_LL, errors_LL =  regr_LL.fit(x_, Y, alpha = 0.01, epsylon = 0.01, max_steps = 10000, Rtype = "LL")

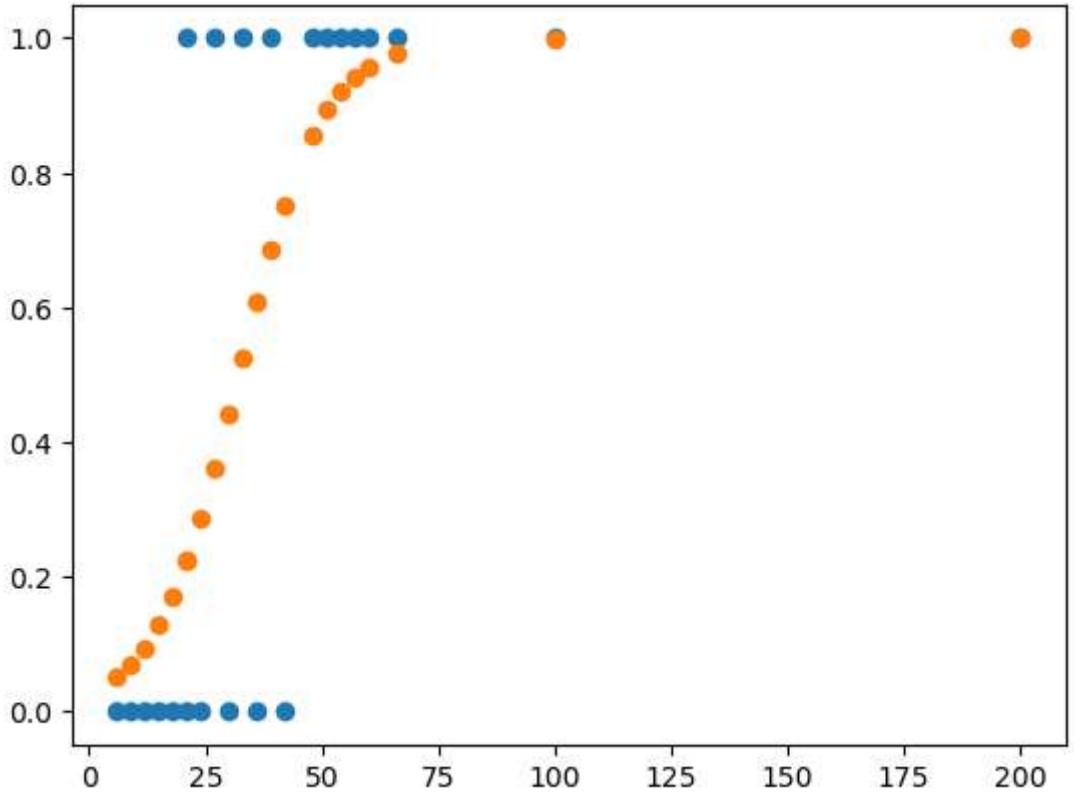
display(f"Время обучения: {time.time() - start_time_LL} сек.")
```

```
'Время обучения: 51.10913443565369 сек.'

'Время обучения: 54.79919672012329 сек.'
```

In [118]:
```
Y_Pred = regr_CE.predict(x_)
plt.scatter(x, Y)
plt.scatter(x, Y_Pred)
```

Out[118]: <matplotlib.collections.PathCollection at 0x1c608958a50>

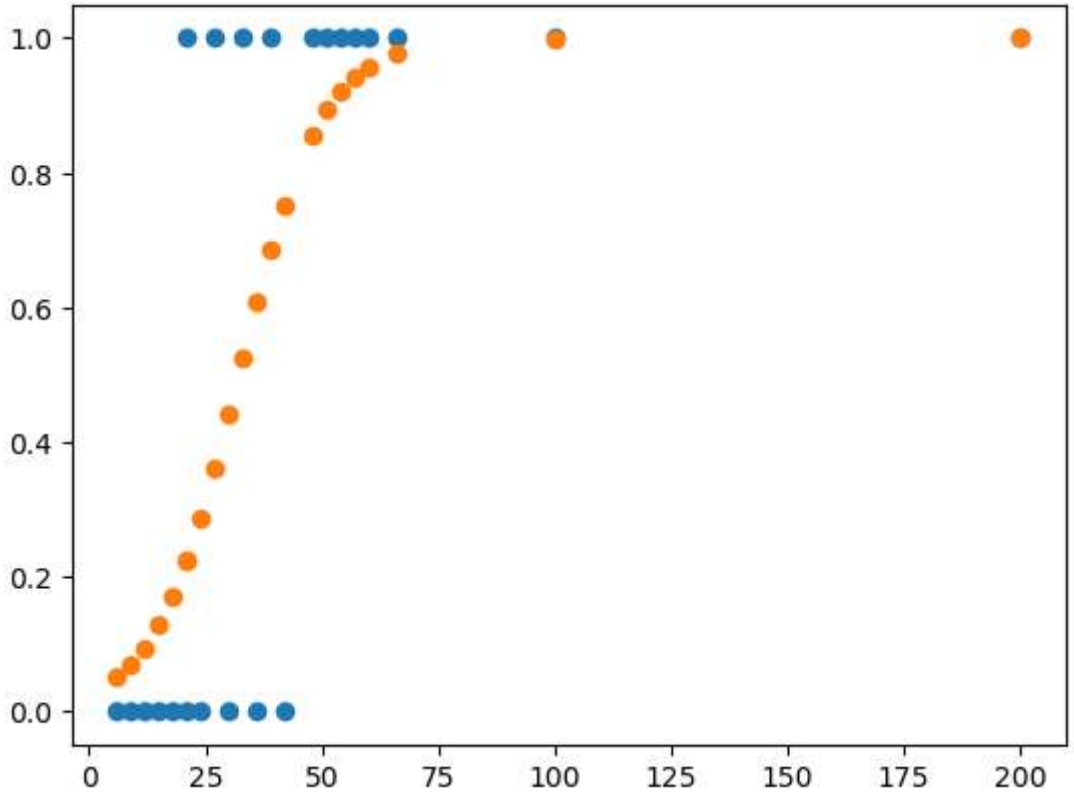


In [126]:
```
display(regr_CE.coefs())
#display(x_)
```

	0
0	-3.598810
1	0.112152

In [120]:
```
Y_Pred = regr_LL.predict(x_)
plt.scatter(x, Y)
plt.scatter(x, Y_Pred)
```

Out[120]: <matplotlib.collections.PathCollection at 0x1c60ed3b550>

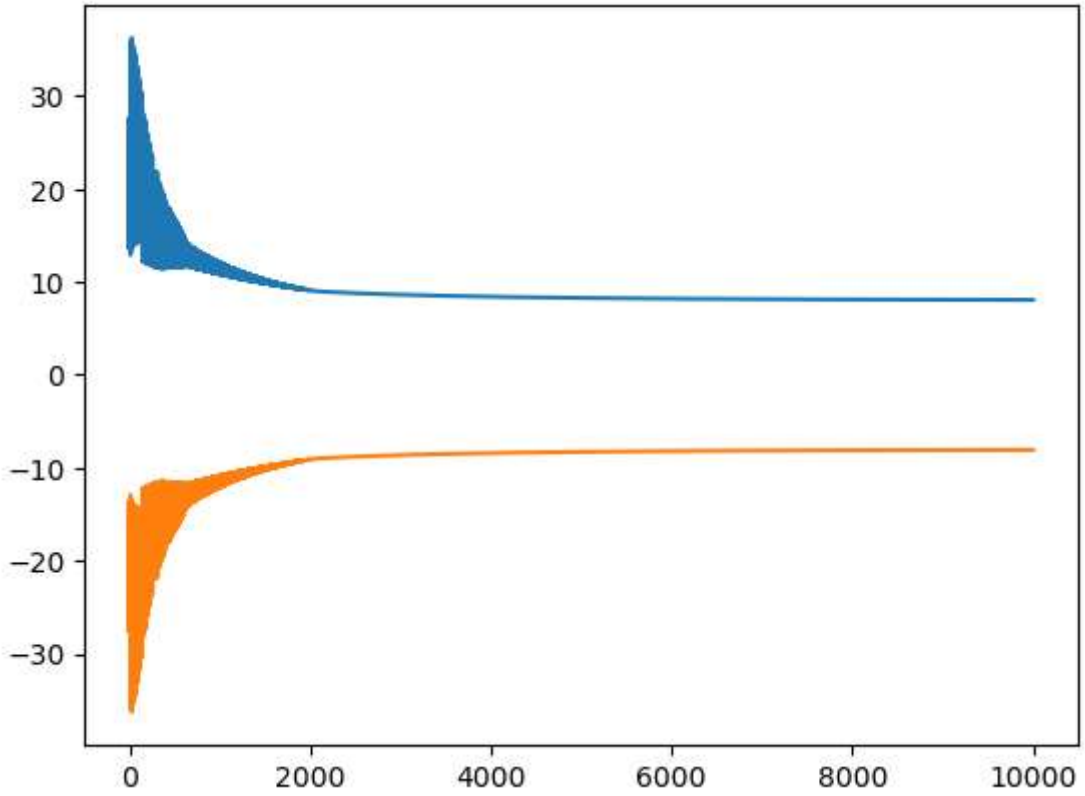


In [125]:
```
display(regr_LL.coefs())
#display(x_)
```

	0
0	-3.598810
1	0.112152

In [122]:
```
plt.plot(steps_CE, errors_CE)
plt.plot(steps_LL, errors_LL)
```

Out[122]:
```
[<matplotlib.lines.Line2D at 0x1c610a47690>]
```



In [123]:
```
print(regr_CE.accuracy(x_, Y))
print(regr_LL.accuracy(x_, Y))
```

```
0    0.818182
dtype: float64
0    0.818182
dtype: float64
```

Реализовать расчеты способами а и б для набора данных из файла insclass_train.csv/ Для 151 406 договоров страхования транспортных средств известны значения ряда признаков, в том числе пол, возраст, стаж вождения и коэффициент бонус-малус водителя, тип, марка, модель, год выпуска, страна – производитель, мощность и объем двигателя, а также признак

target, равный 1, если заключение договора с клиентом является рисковым, и 0 в противном случае (файл insclass_train.csv).

Требуется построить модель, предсказывающую значение признака target для 22 624 договоров из тестового набора данных (файл insclass_test.csv).

В обучающем наборе данных для каждого договора известны следующие поля:

variable_1 - агрегированный коэффициент бонус-малус (повышающий или понижающий стоимость полиса в зависимости от аварийности в предыдущие периоды); variable_2 - индикатор расторжения договора по инициативе страхователя (клиента); variable_3 - индикатор расторжения договора по инициативе страховщика (страховой компании); variable_4 - идентификатор года выпуска транспортного средства; variable_5 - идентификатор страны - производителя транспортного средства; variable_6 - мощность двигателя в лошадиных силах; variable_7 - объем двигателя в куб. см; variable_8 - идентификатор стороны расположения руля (левый или правый); variable_9 - пробег транспортного средства, покрываемый гарантией производителя; variable_10 - индикатор действия гарантии на транспортное средство; variable_11 - "мультидрайв" - индикатор допуска к управлению транспортным средством более одного водителя; variable_12 - возраст транспортного средства (в мес.); variable_13 - возраст водителя с максимальным стажем; variable_14 - коэффициент возраст-стаж; variable_15 - коэффициент краткосрочности; variable_16 - коэффициент мощности; variable_17 - коэффициент "мультидрайв"; variable_18 - территориальный коэффициент; variable_19 - коэффициент "КНДР"; variable_20 - идентификатор канала продаж; variable_21 - марка транспортного средства; variable_22 - модель транспортного средства; variable_23 - индикатор отечественных транспортных средств; variable_24 - пол водителя с максимальным коэффициентом "возраст-стаж"; variable_25 - индикатор пролонгации; variable_26 - индикатор совпадения собственника транспортного средства и водителя; variable_27 - стаж водителя с максимальным коэффициентом "возраст-стаж"; variable_28 - тип транспортного средства; target - класс риска, равный 1, если заключение договора с клиентом является рисковым, и 0 в противном случае.

In [ ]: